On-Chain Digital Inheritance

Author: aernoud@inheritor.app

Website: www.inheritor.app

Date: September 29, 2025

Abstract

The growth of (tokenized) digital assets — from cryptocurrencies, gold ownership recorded as tokens, and social media accounts to personal email accounts, repositories, photos, and websites — has created a new inheritance challenge: ensuring that beneficiaries can access these assets securely and only under the rightful conditions. Without such safeguards, digital property may be lost forever upon a testator's unexpected death or incapacitation.

Inheritor addresses this by introducing on-chain digital inheritance: a blockchain-based execution layer that automates time-locked, event-driven transfer of rights. Inheritor enforces ownership for purely digital assets, while for registry-bound (non-tokenized) assets requiring deeds, it provides a secure channel to release legally recognized instruments.

Core Contribution: Inheritor introduces a fundamentally new class of time-lock mechanism: the architectural time-lock.

Unlike traditional solutions that rely on withholding cryptographic keys or secret shares ("enforcement through scarcity"), Inheritor provides beneficiaries with all cryptographic material from day one—yet decryption remains impossible until smart contract conditions are met. The Inheritor iOS application achieves true infrastructure independence using only the blockchain.

By combining smart contract execution, quantum-resistant encryption (X-Wing hybrid KEM), and architectural enforcement through platform security, Inheritor establishes a durable, censorship-resistant framework for passing on digital property. It reframes inheritance not as document storage but as programmable, verifiable transfer—a model designed to remain accessible across platforms, jurisdictions, and decades, while eliminating traditional trust requirements through architectural guarantees.

Introduction

In today's digital era, individuals accumulate a wide range of assets. Traditional inheritance processes involve creating a will with legal professionals, often including notaries, who act as custodians of confidential documents and transfer deeds. Yet this model struggles to address the growing class of purely digital assets, where the decisive fact is simply who has access.

This mismatch leaves many people unprotected: they postpone creating wills, face difficulty accessing trustworthy notaries, or risk complete loss of digital property through secrecy of keys and passwords. Inheritor directly addresses this gap.

Our approach recognizes three broad categories of assets:

1. **Purely digital assets where access = ownership.**
2. **Registry-bound assets where the law mandates a formal deed (e.g., real estate).**
3. **Personal or hybrid assets where the testator's clear instructions should be respected (e.g., a car, computer, or family keepsake).**

Inheritor is built for categories 1 and 3 as its natural domain, while still supporting category 2 by releasing lawyer/notary-authenticated deeds when required. In all cases, execution shifts from paper-based, human-administered steps to cryptographically enforced, on-chain automation.

Contrasting Existing Approaches

To understand Inheritor's architectural innovation, it's essential to examine how it differs from existing conditional access mechanisms:

- **Escrow and Custodial Models:** Traditional solutions rely on trusted third parties (banks, legal firms, or specialized services) to hold assets until conditions are met. These approaches are fundamentally centralized, introducing single points of failure, ongoing operational costs, and requiring trust in institutions that may face regulatory changes, bankruptcy, or operational failures. (e.g. Casa Covenant — attorney & Casa key custody)
- **Timelock Puzzles and VDFs:** Cryptographic time-lock mechanisms like RSA time-lock puzzles and Verifiable Delay Functions create resource-bound delays based on computational work. While mathematically elegant, they are inherently inflexible—the delay is fixed at creation time and cannot adapt to external conditions like the testator's actual status or changing circumstances. (e.g. mechanisms catalogued in Buka Corner's 2025 survey)
- **Multisig and Social Recovery:** These approaches distribute control among multiple parties, relying on human coordination and social trust. They are human-trust-bound, requiring beneficiaries to coordinate with other keyholders, creating potential for disputes, coercion, or unavailability of required signatories. (e.g. Safe Haven Inheriti — Shamir's Secret Sharing; Vault12 — guardian networks; Casa Covenant — multisig with attorney involvement)
- **DIY Solutions and Custom Hacks:** People often devise creative but fragile workarounds: hardware wallets hidden in safes with handwritten seed phrases, passwords in sealed envelopes with lawyers, USB drives in safety deposit boxes, or encrypted files with keys split among family members. These approaches are fundamentally secrecy-bound and discovery-dependent. They fail catastrophically when physical items are lost, damaged, or stolen; when written instructions become outdated or unclear; when the testator dies suddenly without sharing the location; or when technology changes make the storage medium obsolete. Even sophisticated setups like hardware wallets become mere physical tokens that must still be found, understood, and properly accessed by beneficiaries who may lack technical knowledge. (e.g. Sarcophagus — dead-man switch with Arweave storage)

In contrast, Inheritor creates on-chain policy-bound conditional access. A smart contract serves as an autonomous oracle of the testator's status, with conditions evaluated programmatically against verifiable on-chain activity. No human intervention is required for execution, no fixed delays constrain adaptation, and no trusted custodians introduce centralization risks.

This fundamental difference—binding conditional access to smart contract–based, verifiable policies rather than fixed delays, human coordination, or institutional trust—enables Inheritor's unique combination of trustlessness, flexibility, and decentralization.

How Inheritor Differs: Comparison Summary

Approach	Trust Model	Key Limitation	Access Control
Escrow/Custodial	Institutional Trust	Centralized, single point of failure	Human-administered
Timelock Puzzles/VDFs	Computational Work	Fixed delays, resource-bound	Mathematical difficulty
Multisig/Social Recovery	Human Coordination	Trust in co-signers, coordination risk	Social consensus
DIY / Custom Hacks	Physical secrecy (notes, safes, ad-hoc storage)	Fragile, easily lost, stolen, or forgotten; no verifiability	Manual discovery by heirs
Inheritor	On-Chain Policy	None—programmable & adaptive	Autonomous smart contract

Market Context

The opportunity for on-chain digital inheritance must be seen in the context of the largest wealth transfer in history: $84 trillion globally through 2045, with Europe alone accounting for $3.5 trillion by 2030. The European estate planning market is valued at €3.85 billion today and projected to nearly double by 2031, with the digital segment growing at over 13% annually. Yet despite this growth, more than two-thirds of adults have no formal will, and €300+ billion in inefficiencies persist each year from cross-border disputes, probate delays, and digital asset losses (sources in internal market analysis).

Inheritor directly addresses these inefficiencies. Where traditional notarial and probate systems impose high costs, long delays, and jurisdictional fragmentation, Inheritor offers a cryptographically enforced, cross-border solution that integrates with existing legal frameworks. By aligning with regulatory clarity in Europe (GDPR, MiCA) and leveraging blockchain's immutability, Inheritor is positioned at the intersection of demographic inevitability and digital transformation.

Defining On-Chain Digital Inheritance

On-Chain Digital Inheritance moves beyond digitizing paper wills. It reconceptualizes inheritance as a cryptographically enforced, autonomous transfer of ownership. Smart contracts grant beneficiaries provable, time-locked access to digital or tokenized tangible assets, executing conditions on-chain. Legal professionals remain involved only where formal instruments are required, so Inheritor integrates with—rather than replaces—existing frameworks.

This model is broader than cryptocurrency or files. Through digitization — and, where supported, tokenization — it can manage rights to real-world assets (e.g., real estate titles, IP, share certificates) while excelling where access itself equals ownership. The system's tamper-proof, transparent, decentralized foundation aligns with global e-records and signature regimes (eIDAS, ESIGN/UETA).

This peer-to-peer model lets individuals appoint beneficiaries directly, eliminating reliance on custodians and avoiding the risks of entrusting private keys to intermediaries. Existing solutions—multisig setups, private servers—still depend on institutional trust; by contrast, blockchains provide a trustless, distributed substrate that meaningfully disintermediates the traditional notary model for digital assets.

Smart contracts form the execution layer: they transfer rights under predefined, verifiable conditions (e.g., within one month after the inheritor's death), with cryptographic assurances against tampering. A dead-man switch pattern can trigger the state transition that makes an inheritance claimable. By storing ciphertext and policy references on-chain (with encrypted payloads on decentralized storage) and controlling decryption via contract-checked conditions, Inheritor enables secure, conditional transfer without manual administration by an executor in the execution layer.

The Challenge of Digital Inheritance

Traditional inheritance frameworks were designed for physical property and paper records. They assume assets are mediated by registries, banks, or custodians who can be instructed to release them after death. Digital-native assets break this model.

Unexpected death or incapacitation can lead to the permanent loss of digital assets, as security protocols discourage sharing passwords and private cryptographic keys. Without access to these credentials, beneficiaries cannot retrieve wallets, accounts, or encrypted archives. Unlike property deeds, there is often no central registry or institution to reissue ownership—if the key is lost, the asset is gone forever.

Trustees and executors are also reluctant to handle digital assets:

- **Liability risk:** Possessing private keys effectively makes them custodians. Any mistake, loss, or theft could expose them to personal liability.
- **Instant mutability:** Digital assets can be transferred or drained in seconds. A trustee who holds a key cannot guarantee the estate will remain intact until probate is completed.
- **Ongoing change:** Passwords are rotated, private keys are re-generated, photos and documents are constantly added, accounts closed or migrated. A static snapshot (USB drive, printed seed phrase, sealed envelope) becomes outdated almost immediately, making "store and hand over" approaches unreliable.
- **Legal uncertainty:** Many jurisdictions still lack clear rules for fiduciaries managing digital wallets or accounts, leaving professionals unwilling to accept responsibility.

This creates a fundamental paradox: assets must remain inaccessible until the testator's death, yet modern cryptographic systems require giving beneficiaries the keys up front—allowing premature access.

To resolve this, inheritance must move beyond traditional key-sharing. We need systems that enforce conditional access without exposing secrets prematurely. Blockchain technology, with its immutable state and autonomous smart contracts, provides this foundation. By combining smart contracts with encryption and decentralized storage, Inheritor creates a trustless, time-locked inheritance mechanism that ensures assets remain secure during life but accessible at the right moment after death.

Blockchain Platform Strategy: Ethereum L1 and Arbitrum L2

After evaluating various options, we decided to build Inheritor on a dual-layer strategy utilizing both Ethereum Layer 1 and Arbitrum One. This approach recognizes that different inheritance scenarios have different security and cost requirements, allowing users to optimize their digital inheritance strategy based on their specific needs.

However integrating Inheritor with the Cardano blockchain is on our roadmap.

Ethereum Layer 1: Maximum Security for High-Value Estates

Working directly on Ethereum's Layer 1 provides optimal security, decentralization, and censorship resistance, which are crucial for high-value inheritance plans requiring absolute trust. By leveraging Ethereum's Layer 1, we ensure maximum security, decentralization, and longevity for the smart contracts governing digital inheritance.

While transacting with smart contracts on Ethereum's Layer 1 can be relatively costly due to higher gas fees, we consider this expense to be a worthwhile investment in security and trustlessness for significant estates. The additional costs are easily offset when compared to the operational expenses and risks associated with centralized solutions like notaries or solutions acting as or depending on third-party custodians.

Arbitrum One: Cost-Efficient Alternative for Smaller Estates

Arbitrum One provides a pragmatic, low-cost alternative for users prioritizing affordability. By leveraging Ethereum's security through Optimistic Rollups, Arbitrum slashes transaction fees by over 90%, making it ideal for smaller estates or frequent updates to inheritance plans.

However, Arbitrum's security model introduces trade-offs. Its validation process relies on a smaller set of validators to detect and contest fraudulent transactions during a 7-day dispute window. While Ethereum ultimately enforces finality, beneficiaries must temporarily trust that validators act diligently during this period. Additionally, Arbitrum's sequencer could theoretically delay or exclude transactions, though mitigations like forced inclusion via Ethereum exist.

Inheritor's integrated approach allows users to maintain their Digital Will on either blockchain, tailoring their inheritance strategy to optimize for uncompromising security or cost efficiency based on estate value and risk tolerance.

Digital Asset Storage with Arweave

Having established our blockchain platform strategy, we now address the practical challenge of storing large digital assets in a permanent, decentralized manner. Storing large digital assets (blobs) directly on the Ethereum blockchain is impractical due to inherent limitations. Ethereum imposes high gas costs for data storage, rendering large file uploads economically unfeasible. The network enforces strict block size and transaction data limits to maintain performance and decentralization.

Arweave offers significant advantages for secure, long-term inheritance solutions. It is a decentralized, append-only storage network utilizing a blockchain-like structure called the Blockweave. Its distributed nature ensures data redundancy and resistance to censorship or data loss.

The append-only architecture guarantees immutability; once data is stored, it cannot be altered or deleted. This is crucial for inheritance, as it ensures the digital asset remains intact and untampered until the beneficiary accesses it.

Moreover, Arweave's fee structure significantly enhances the practicality of Inheritor for digital inheritance. Arweave operates on a one-time payment model that ensures permanent data storage, eliminating the need for recurring fees. This "pay once, store forever" approach perfectly aligns with the requirements of digital inheritance, where assets must remain accessible and intact indefinitely. By leveraging Arweave, Inheritor provides a cost-effective solution that guarantees the enduring preservation of digital assets without the burden of ongoing operational expenses. This ensures that beneficiaries can reliably access their inherited assets long into the future, without concerns about storage sustainability or escalating costs.

Inheritor in tandem with Arweave employs Merkle trees to handle large file uploads efficiently and to verify data integrity. By breaking files into chunks and hashing them into a Merkle tree structure, it enables quick detection of any tampering attempts. This cryptographic method ensures that even with large datasets, the integrity verification process remains efficient.

Overcoming Blockchain Challenges

While Ethereum and Arbitrum provide the execution layer for inheritance, and Arweave ensures permanent storage, several fundamental blockchain characteristics present unique challenges for our use case.

Challenge 1: Preventing Premature Access

Blockchains are public and transparent; all data and transactions are visible. In the context of inheritance, this creates a critical paradox: if the beneficiary already holds the private key, then publishing the ciphertext and all encryption material on-chain would allow them to unlock the inheritance immediately, bypassing any intended conditions. Once a decryption key is disclosed, access can never be delayed again. The key must therefore remain inert until the rightful time.

Inheritor resolves this by binding key usability—not key possession—to smart contract state, introducing a cryptographically verifiable time-release mechanism. This ensures inheritance assets remain inaccessible during the testator's lifetime but become claimable under predefined, on-chain conditions without relying on secrecy or custodianship.

Challenge 2: Simplifying User Interaction

Interacting with blockchains is daunting for most users: wallets, private keys, and gas fees create friction. Inheritor removes this complexity by using ERC-4337 smart accounts (account abstraction). Users don't need to create EOAs or handle raw keys; the app mediates signatures and policy while preserving self-custody.

With account abstraction, users approve actions using familiar iOS authentication (e.g., Face ID). Under the hood, the app signs and submits an ERC-4337 UserOperation via a bundler, and gas is covered by a paymaster. Inheritor funds these fees and recovers costs through StoreKit Consumable in-app purchases—so users never need to hold ETH just to transact.

While Inheritor leverages ERC-4337 account abstraction with Paymaster-subsidized gas for a seamless mobile experience, the underlying smart contracts also remain directly accessible via externally owned accounts, ensuring resilience and infrastructure independence.

By abstracting keys and gas behind a standard AA flow, Inheritor makes on-chain actions feel like any other iOS interaction, while still providing verifiable, on-chain execution.

Cryptography Without the Crypto

For the encryption and decryption of digital assets, Inheritor uses a dual-key cryptographic architecture that separates blockchain interaction from asset encryption. This design ensures compatibility with today's infrastructure while providing quantum-safe protection for the future.

Because Arweave stores data permanently and publicly, all digital assets are encrypted with post-quantum security, while classical cryptography is maintained only where required for blockchain operations. The roles are clearly divided: secp256k1 keys handle Ethereum functions such as signing transactions, executing smart contracts, and verifying ownership, while X-Wing quantum-safe keys protect the digital assets themselves. This means that even if quantum computers eventually compromise ECDSA signatures, inherited assets remain secure.

Asset encryption follows a hybrid model. Each file is encrypted with a randomly generated AES-256 symmetric key. X-Wing then protects only this 32-byte key through quantum-safe encapsulation. The resulting shared secret is expanded with HKDF into a key-encryption key (KEK), which wraps the AES key using AES-256-GCM for authenticated encryption. This approach balances performance and security: AES efficiently handles large data, while X-Wing provides future-proof protection for the key material.

This "implement now for later" philosophy ensures that inheritances created today will remain secure across decades, immune to both quantum threats and the permanence of decentralized storage, while still leveraging Ethereum's existing infrastructure for trustless execution.

Architectural Time-Lock: A New Class of Conditional Access

Inheritor introduces a new enforcement primitive: the architectural time-lock. Unlike traditional methods that rely on withholding cryptographic pieces, computational delays, or trusted custodians, architectural time-locks operate at the client/infrastructure boundary. They enforce conditional access through system design, not cryptographic scarcity.

Traditional methods—such as time-lock puzzles or Verifiable Delay Functions (VDFs)—depend on mathematical difficulty or fixed computation, which are rigid, resource-intensive, and unresponsive to real-world events. Similarly, existing digital inheritance solutions like Sarcophagus, Safe Haven, and Casa Covenant all rely on "enforcement through scarcity": some critical piece is always withheld or delayed to prevent early decryption—whether it's cryptographic keys, secret shares, or computational solutions. The decryption key or capability is made scarce or unreachable until the right moment.

Inheritor flips this paradigm with the insight that enforcement doesn't require withholding the cryptographic material itself. In Inheritor's design, everything needed to decrypt is already in the hands of the heir from the start—yet decryption is impossible until the correct moment. All cryptographic pieces (private key and ciphertext) are present with the beneficiary from day one, but enforced inert. Inheritor ties decryption strictly to the state of a smart contract, with conditions like Designated → Claimable → Claimed enforced through the immutable, censorship-resistant blockchain.

This model creates a security paradigm where possession of cryptographic material does not equal authorized access. Instead, policy is enforced through architectural design combined with smart contract verification. Inheritor implements this principle through two complementary paths: one leveraging platform security guarantees for iOS users, and another using minimal infrastructure for cross-platform compatibility. Both paths enforce identical on-chain policies while optimizing for different use cases.

iOS Implementation: Pure Infrastructure Independence

The iOS implementation represents the architectural time-lock in its purest form, leveraging iOS 26's X-Wing CryptoKit implementation where private keys are inherently non-exportable at the API level. This creates a sharp distinction between application-level and system-level key isolation.

In typical designs, one might store BIP32 or other private keys in the iOS Keychain and simply withhold an export UI—a policy decision within the application layer that can often be bypassed by reverse engineering, memory dumps, or malicious code paths. In contrast, iOS 26's X-Wing implementation provides system-level enforcement: keys can be backed up via iCloud Keychain for resilience, yet they remain architecturally bound to Apple's platform with no API for extracting raw key material. This transforms time-locked inheritance from a software policy ("don't expose export") into a de facto OS/hardware-backed guarantee ("you physically cannot export").

The beneficiary's X-Wing private key resides securely on-device and is architecturally non-exportable, while everything else required for decryption—encrypted ciphertext, wrapped AES key, KEM capsule, nonces, and authentication tags—is stored publicly on Arweave. Beneficiaries hold their inheritance keys from day one; however, these keys exist in a form that cannot be extracted—they can only be used through authorized API calls.

Despite possessing their private key and all other decryption components being publicly accessible, decryption remains architecturally blocked until the app verifies the inheritance is Claimable according to the smart contract. The Inheritor app verifies smart contract state (Designated → Claimable) before instructing iOS to perform decryption operations. The enforcement emerges from the combination of non-exportable keys (platform security) and application logic that gates decryption operations behind blockchain state verification. The scarcity is not the absence of a key, but the inability to make the key function until the smart contract allows.

This architecture eliminates dependence on external infrastructure services entirely. The iOS application can decrypt inheritance assets using only blockchain state verification and public data from Arweave, representing complete infrastructure independence: iOS beneficiaries enjoy true peer-to-peer operation with only Ethereum and Arweave dependencies.

ML-KEM Implementation: Cross-Platform Portability

The ML-KEM path addresses a different challenge: ensuring inheritance accessibility across platforms and through external tools. In this implementation, keys are portable by design, making them valuable for cross-platform compatibility, but this portability means that purely client-side applications cannot enforce the time-lock. Once portable keys are distributed, and since all other components are already public on Ethereum and Arweave, decryption would always be possible without additional safeguards.

To solve this, Inheritor introduces a minimal conditional release service. In our current implementation this is a Cloudflare Worker, but the design is agnostic: any lightweight, auditable service can fulfill the role. Its sole function is to release the wrapped symmetric key only when the Ethereum smart contract confirms that the inheritance is Claimable and the beneficiary has authenticated via EIP-712 signatures. This approach maintains cryptographic time-lock enforcement while enabling the portability essential for external tools and alternative client implementations.

This trade-off—requiring minimal infrastructure for maximum compatibility—ensures that external tool users benefit from portable, standards-based access that works across platforms with appropriate infrastructure support.

By implementing architectural enforcement through two complementary mechanisms, Inheritor establishes a new category of policy-bound conditional access: one that provides both infrastructure independence and cross-platform compatibility, leverages blockchain as the single source of truth, and remains adaptable across decades of technological change.

Resilience Through Dual-Path Design

Inheritor deliberately implements both architectural paths because neither alone provides adequate long-term resilience. While each path can technically enable digital inheritance, only the dual-path approach ensures the system survives platform changes, corporate decisions, and technological evolution over the multi-decade timeframes typical of inheritance.

The iOS path represents the ideal: maximal user experience with zero infrastructure dependencies, complete peer-to-peer operation using only Ethereum and Arweave. However, this path remains subject to app store policies and platform availability. The ML-KEM path serves as essential insurance against vendor lock-in, ensuring that inheritance remains accessible regardless of any single company's decisions or platform changes.

Crucially, both paths enforce identical on-chain policies through different mechanisms—the iOS app through architectural enforcement, the ML-KEM path through conditional release. Both are bound to the same smart contract conditions and the same context binding (HKDF info + AES-GCM AAD), ensuring consistent security policy regardless of access method.

This dual-path approach achieves true censorship resistance through redundancy, not through any single application. Even if Apple or any vendor discontinues support, the format and enforcement logic remain public and independently implementable. The system's decentralization derives from the open asset format and the existence of alternate implementation paths, ensuring inheritance accessibility across decades of technological change.

Exfiltration Resistance Through Architectural Constraints

The architectural design of Inheritor creates multiple independent barriers against data exfiltration, even by the application developer. These constraints emerge from the interaction between iOS platform security, public blockchain storage, and verifiable data formats.

Non-Exportable Private Keys

The foundation of exfiltration resistance lies in iOS 26's X-Wing implementation, where XWingMLKEM768X25519.PrivateKey provides no API for extracting raw key material. This means that even the Inheritor application—regardless of any malicious code—cannot access or transmit private keys. Beneficiaries generate their own keypairs on their devices, and these keys remain architecturally bound to the user's Apple ecosystem through iCloud Keychain. This transforms the traditional threat model: rather than trusting developers not to steal keys, the platform ensures keys cannot be stolen.

Complete Transparency in the X-Wing Path

For iOS users, the entire inheritance system operates through public channels. All cryptographic materials required for decryption exist on-chain: encrypted assets on Arweave and inheritance metadata on Ethereum. The claim process requires zero private communication—no API calls to developer-controlled servers, no hidden data channels. This transparency is easily verifiable through network monitoring: the iOS app communicates only with Ethereum and Arweave during the claim process. Any deviation from this pattern would be immediately detectable.

Constrained and Verifiable ML-KEM Path

While the ML-KEM path necessarily includes a private channel to store wrapped symmetric keys, this channel is architecturally constrained. The data format stored in Cloudflare exactly matches the structure used by the public reference implementation (Beneficiary_Claim.js). Network analyzers can verify round-trip integrity: what is sent to Cloudflare during inheritance creation must be identical to what is retrieved during the claim process. Any server-side additions or modifications would be immediately detectable. The wrapped key has a predictable size based on ML-KEM-768 specifications, and any attempt to exfiltrate additional data would require deviating from this format, making it observable through traffic analysis.

The result is a system where developer malfeasance is either impossible (key extraction), unnecessary (X-Wing path fully public), or detectable (format violations or server-side tampering). This represents a fundamental advancement over traditional cryptographic applications where users must simply trust that developers won't include backdoors. In Inheritor, the architecture itself provides the security guarantees, with violations being either prevented by the platform or observable through network analysis.

Conclusion

Decentralization Through Design. Inheritor achieves true decentralization not through any single application or distribution channel, but through its fundamental architecture: state and data exist on Ethereum + Arweave, and multiple clients can enforce identical on-chain policies through different mechanisms. The iOS app is a convenience client; censorship resistance derives from the open asset format, standardized protocols, and the existence of alternate clients—not from app store policies or single points of distribution. Even if Apple or any single vendor discontinues distribution, the format and enforcement logic remain public and independently implementable, ensuring the system remains fully functional through alternative implementations of the same open standards.

Inheritor introduces a new paradigm: On-Chain Digital Inheritance as a programmable and autonomous system of transfer. Its natural strength lies in assets where access equals ownership, while also enabling personal instructions to be carried out with cryptographic certainty. For registry-bound assets requiring formal deeds, Inheritor functions as the secure release layer that complements legal processes.

The Path to Frictionless Inheritance: Inheritor's most practical achievement may be how it enables the digitization of real-world inheritances—making property deeds, vehicle titles, and personal instructions flow to beneficiaries with the same programmable ease as transferring cryptocurrency. Where traditional inheritance requires executors, notaries, physical documents, and coordination across multiple parties, Inheritor reduces complex asset transfers to the same reliability and simplicity as blockchain-native tokens. A house deed encrypted within Inheritor transfers as smoothly as ETH; family heirlooms with detailed instructions move as seamlessly as an NFT. This represents the true democratization of inheritance: transforming historically complex, paper-dependent processes into automated, verifiable transfers that execute with digital precision.

Inheritor represents a significant advancement in how individuals can confidently and securely pass on their most valuable rights — across generations, across jurisdictions, and across time.

References

1. **Ethereum & ERC Standards**
   - Ethereum Whitepaper: A Next-Generation Smart Contract and Decentralized Application Platform. Vitalik Buterin. Available at: https://ethereum.org/en/whitepaper/
   - ERC-4337: Account Abstraction Using Alt Mempool. Available at: https://eips.ethereum.org/EIPS/eip-4337
   - EIP-712: Typed structured data hashing and signing. Available at: https://eips.ethereum.org/EIPS/eip-712
2. **Arbitrum**
   - Arbitrum Documentation. Available at: https://docs.arbitrum.io/
   - Arbitrum Nitro Whitepaper. Available at: https://github.com/OffchainLabs/nitro/blob/master/docs/Nitro-whitepaper.pdf
3. **Arweave**
   - Arweave Whitepaper: A Protocol for Economically Sustainable Information Permanence. Available at: https://www.arweave.org/whitepaper.pdf
   - Arweave Documentation. Available at: https://docs.arweave.org/
4. **Cryptography**
   - AES-GCM: NIST Special Publication 800-38D. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. Available at: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
   - HKDF: RFC 5869. HMAC-based Extract-and-Expand Key Derivation Function. Available at: https://tools.ietf.org/html/rfc5869
   - BIP-39: Mnemonic code for generating deterministic keys. Available at: https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki
   - BIP-32: Hierarchical Deterministic Wallets. Available at: https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki
   - secp256k1: SEC 2: Recommended Elliptic Curve Domain Parameters. Available at: https://www.secg.org/sec2-v2.pdf

5. **Post-Quantum Cryptography**
   - NIST Post-Quantum Cryptography Standardization. Available at: https://csrc.nist.gov/projects/post-quantum-cryptography
   - ML-KEM (FIPS 203): Module-Lattice-Based Key-Encapsulation Mechanism Standard. Available at: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf
   - X-Wing: A Hybrid KEM Combining X25519 and ML-KEM-768. Available at: https://eprint.iacr.org/2024/039
   - Understanding X-Wing. Available at: https://words.filippo.io/dispatches/xwing/
6. **iOS & Swift**
   - iOS Security Guide. Apple Inc. Available at: https://support.apple.com/guide/security/welcome/web
   - CryptoKit Documentation. Available at: https://developer.apple.com/documentation/cryptokit
   - X-Wing in CryptoKit (iOS 26+). Available at: https://developer.apple.com/documentation/cryptokit/xwingmlkem768x25519
   - Swift Concurrency Documentation. Available at: https://docs.swift.org/swift-book/LanguageGuide/Concurrency.html
7. **Legal & Regulatory**
   - Regulation (EU) No 910/2014 (eIDAS). Available at: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=uriserv%3AOJ.L_.2014.257.01.0073.01.ENG
   - Electronic Signatures in Global and National Commerce Act (E-SIGN). Available at: https://www.govinfo.gov/content/pkg/PLAW-106publ229/pdf/PLAW-106publ229.pdf
   - Uniform Electronic Transactions Act (UETA). Available at: https://www.uniformlaws.org/committees/community-home?CommunityKey=2c04b76c-2b7d-4399-977e-d5876ba7e034
   - General Data Protection Regulation (GDPR). Available at: https://eur-lex.europa.eu/eli/reg/2016/679/oj
   - Markets in Crypto-Assets Regulation (MiCA). Available at: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32023R1114
8. **Account Abstraction Infrastructure**
   - Biconomy SDK Documentation. Available at: https://docs.biconomy.io/
   - Pimlico Bundler Documentation. Available at: https://docs.pimlico.io/
   - Account Abstraction (ERC-4337) Infrastructure Overview. Available at: https://www.erc4337.io/

9. **Solidity & Smart Contract Development**
   - Solidity Documentation. Available at: https://docs.soliditylang.org/
   - OpenZeppelin Contracts. Available at: https://docs.openzeppelin.com/contracts/
   - Ethereum Smart Contract Best Practices. ConsenSys. Available at: https://consensys.github.io/smart-contract-best-practices/
10. **Web3 Libraries**
   - web3.js Documentation. Available at: https://web3js.readthedocs.io/
   - web3.swift Documentation. Available at: https://github.com/argentlabs/web3.swift
11. **Cloudflare Workers**
   - Cloudflare Workers Documentation. Available at: https://developers.cloudflare.com/workers/
   - Workers KV Documentation. Available at: https://developers.cloudflare.com/workers/runtime-apis/kv/
12. **Ethers.js**
   - Ethers.js Documentation. Available at: https://docs.ethers.io/v5/
   - GitHub Repository: ethers-io/ethers.js. Available at: https://github.com/ethers-io/ethers.js/

Acknowledgments

We would like to thank the open-source and AI community and the developers of the technologies utilized in this project. Their contributions have been invaluable in enabling the development of Inheritor.

Future Work

Looking ahead, we plan to:

- **Expand Blockchain Compatibility:** Integrate Inheritor with Cardano to offer users more options based on their preferences and needs.
- **Enhance User Experience:** Continue simplifying the user interface to make the process of setting up digital inheritance plans even more accessible to non-technical users.
- **Security Audits:** Undergo comprehensive security audits by third-party experts to validate and enhance the security measures implemented.
- **Legal Compliance:** Collaborate with legal experts to ensure that digital inheritance created with Inheritor is recognized and enforceable under various jurisdictions.

Glossary

- **Smart Contract:** A self-executing contract with the terms of the agreement directly written into code, running on a blockchain.
- **Blockchain Trilemma:** The challenge in achieving decentralization, security, and scalability (or privacy) simultaneously in blockchain systems.
- **Dead-Man Switch:** A mechanism that triggers an action if a required input is not received within a set time, used here to transition the contract state based on the owner's activity.
- **AES-256-GCM:** Advanced Encryption Standard in Galois/Counter Mode — a symmetric encryption algorithm that provides both **confidentiality** (encryption) and **integrity** (authentication) by combining AES encryption with Galois Message Authentication Code (GMAC).
- **AAD (Additional Authenticated Data):** Data that is authenticated but not encrypted in AES-GCM. In Inheritor's context, AAD binds the ciphertext to specific inheritance metadata (contract address, beneficiary role, algorithm version), ensuring the encrypted asset cannot be used in different contexts even if the ciphertext is copied.
- **Post-Quantum Cryptography:** Cryptographic algorithms designed to be secure against attacks by both classical and quantum computers, protecting against future quantum computing threats.
- **ML-KEM (Module Lattice-based Key Encapsulation Mechanism):** NIST's standardized post-quantum key encapsulation mechanism based on lattice cryptography, providing quantum-resistant key agreement.
- **X-Wing:** A hybrid key encapsulation mechanism that combines ML-KEM-768 (post-quantum) with X25519 (classical elliptic curve) to provide security against both classical and quantum computer attacks.
- **X25519:** A high-speed elliptic curve key agreement algorithm based on Curve25519, providing classical cryptographic security and performance.
- **Key Encapsulation Mechanism (KEM):** A cryptographic technique where a sender generates a random symmetric key and encrypts it using the recipient's public key, enabling secure key agreement.
- **Dual-Key Architecture:** Inheritor's design using separate key pairs for blockchain operations (secp256k1) and asset encryption (X-Wing quantum-safe keys), providing role isolation and quantum resistance.
- **Dual-Recipient Encryption:** Inheritor's encryption scheme that creates a single encrypted asset accessible through two different cryptographic paths, optimizing for both iOS applications and external tools.
- **Application-Level Time-Lock:** A security mechanism where time-lock enforcement is implemented through application business logic rather than cryptographic component withholding, enabling infrastructure independence while maintaining conditional access control.
- **Infrastructure Independence:** The ability of an application to operate without dependence on external web services or infrastructure, relying only on Ethereum and Arweave.
- **Arweave Blockweave:** A blockchain-like data structure used by Arweave to store data permanently and efficiently.
- **KV Storage:** Key-Value Storage, a simple database that stores data as a collection of key-value pairs.
- **HKDF:** HMAC-based Key Derivation Function; a method for deriving cryptographically strong keys from a shared secret. It employs a random salt and a domain-specific info string to ensure the derived key is uniformly random and context-bound.
- **Salt (Cryptographic):** A randomly generated value used in key derivation to ensure that even identical inputs produce unique derived keys, preventing vulnerabilities from key reuse.
- **Nonce:** A number used only once in encryption processes, such as in AES-GCM, to guarantee that each encryption operation produces a unique ciphertext even when the same plaintext is encrypted multiple times.
- **Domain Separation (Info String):** A context-specific value used in key derivation (e.g., within HKDF) to bind the derived key to a particular application or purpose, ensuring that keys derived in different contexts remain distinct.
- **Beneficiary Code:** A composite identifier containing both classical (secp256k1) and quantum-safe (X-Wing) public keys along with the beneficiary's name, enabling complete inheritance setup.
- **Executor (Common Law):** The personal representative appointed by a will (or by the court if none is named) to administer the estate, including collecting assets, paying debts/taxes, and distributing the remainder to beneficiaries.
- **Probate (Common Law):** The court-supervised process that authenticates the will (if any), appoints a personal representative, and oversees estate administration and distribution according to law.

Contact Information

For more information about Inheritor or to contribute to the project, please contact:

- Email: aernoud@inheritor.app
- Website: www.inheritor.app

Disclaimer

This white paper is for informational purposes only and does not constitute legal, financial, or investment advice. Users should consult with professional advisors before making any decisions based on the information provided.

Note: The technologies and protocols referenced in this document are subject to ongoing development and improvement. Readers are encouraged to refer to the latest documentation and research for the most up-to-date information.